

Algorithmes et Pensée Computationnelle

Algorithmes de graphes

Le but de cette séance est de comprendre le fonctionnement des graphes et d'appliquer des algorithmes courants sur des graphes simples.

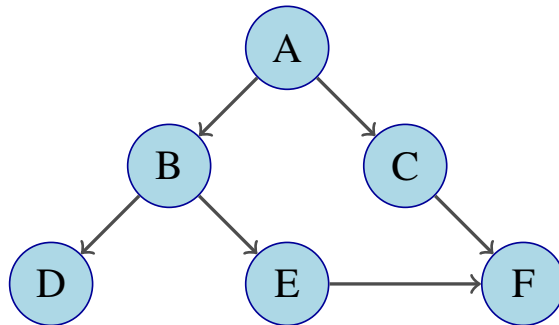
Le code présenté dans les énoncés se trouve sur Moodle, dans le dossier `code`. Le temps indiqué (🕒) est à titre indicatif.

1 Breadth-First Search

Question 1: (🕒 10 minutes) Adjacency list et adjacency matrix : Python

L'adjacency list (ou liste de contiguïté) et l'adjacency matrix (ou matrice de contiguïté) sont les deux méthodes dont nous disposons pour représenter un graphe.

Utilisez ces deux méthodes de représentations pour stocker le graphe ci-dessous dans un programme Python :



Question 2: (🕒 20 minutes) Breadth-First Search algorithm : Python

Nous allons maintenant nous intéresser au premier algorithme portant sur les graphes : **Breadth-First Search**. Le but du Breadth-First Search est de trouver tous les sommets atteignables à partir d'un sommet de départ.

Implémentez l'algorithme en suivant les étapes suivantes :

1. Initialisez:
 - une **file** (queue FIFO — de type liste) qui contient initialement le sommet de départ.
 - une liste (appelée `visited`) de sommets visités initialement vide.
2. Tant que la **file** n'est pas vide, répétez:
 - (a) Retirez le premier sommet de la file et appelez-le *u*.
 - (b) Pour chaque sommet adjacent *v* de *u*:
 - si *v* n'est pas encore dans **visited**, alors :
 - ajoutez *v* à **visited**.
 - insérez *v* à la fin de la **file**.
3. Quand la **file** devient vide, tous les sommets atteignables depuis le sommet de départ ont été visités en largeur.

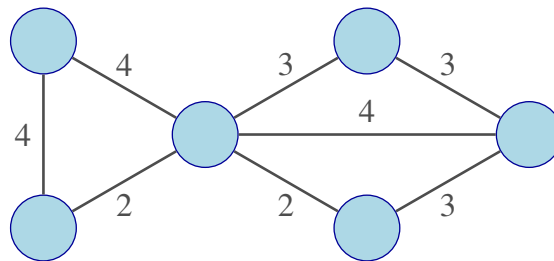
2 Arbre couvrant minimum (ACM) — Minimum Spanning tree (MST)

Nous allons maintenant nous intéresser à l'**algorithme de Kruskal**. Ce dernier s'applique uniquement aux **weighted graphs** (ou graphes pondérés). Ces derniers sont des graphes où les arêtes ont des poids, représentant par exemple une distance. L'algorithme de Kruskal a pour but de trouver un **arbre couvrant minimum** (ACM). Un arbre couvrant minimum S de G est un sous-graphe connexe de G tel que :

1. $V' = V$, c'est-à-dire que tous les sommets de G sont aussi dans S
2. (V', E') ne contient pas de cycle (pas de cycle dans S)
3. S est le graphe satisfaisant (1) et (2) et ayant la plus petite somme des poids
4. Pour chaque paire de sommets dans G , S contient le chemin minimax entre ces deux sommets. Le chemin minimax est un chemin pour lequel le poids de l'arête la plus lourde est minimisé.

Question 3: (🕒 5 minutes) Algorithme de Kruskal: Papier

Appliquez l'algorithme de Kruskal au graphe suivant:



Question 4: (🕒 15 minutes) Algorithme de Kruskal : Python

Vous trouverez ci-dessous l'algorithme de Kruskal implémenté en Python (disponible sur Moodle). Parcourez la fonction **Kruskal_algo(Graph)** afin de vous assurez que vous ayez bien compris le fonctionnement :

```
1 # Question 4
2
3 class Graph:
4     def __init__(self, vertices): # permet de créer un graphe lorsqu'on écrit
        p.ex Graph(6), il faut notamment indiquer le nombre de sommets
5         self.V = vertices
6         self.graph = []
7
8     def add_edge(self, u, v, w): # ajoute une arête entre le sommet u et v
        avec un poids w
9         self.graph.append([u, v, w])
10
11     def find(self, parent, i): # Correspond à la fonction Find-set(x)
        présentée dans le cours
12         if parent[i] == i:
13             return i
14         return self.find(parent, parent[i])
15
16     def apply_union(self, parent, rank, x, y): # Correspond à la fonction
        Union(x,y) présentée dans le cours
17         xroot = self.find(parent, x)
18         yroot = self.find(parent, y)
19         if rank[xroot] < rank[yroot]:
20             parent[xroot] = yroot
21         elif rank[xroot] > rank[yroot]:
22             parent[yroot] = xroot
23         else:
24             parent[yroot] = xroot
25             rank[xroot] += 1
```

```

26
27 g = Graph(6)
28 g.add_edge(0, 1, 4)
29 g.add_edge(0, 2, 4)
30 g.add_edge(1, 2, 2)
31 g.add_edge(1, 0, 4)
32 g.add_edge(2, 0, 4)
33 g.add_edge(2, 1, 2)
34 g.add_edge(2, 3, 3)
35 g.add_edge(2, 5, 2)
36 g.add_edge(2, 4, 4)
37 g.add_edge(3, 2, 3)
38 g.add_edge(3, 4, 3)
39 g.add_edge(4, 2, 4)
40 g.add_edge(4, 3, 3)
41 g.add_edge(5, 2, 2)
42 g.add_edge(5, 4, 3)
43
44 def kruskal_algo(Graph):
45     result = [] # Permettra de stocker le résultat
46     i, e = 0, 0 # Index utilisé dans l'algorithme
47
48     Graph.graph = sorted(Graph.graph, key=lambda item: item[2]) # Trie les
    arêtes par poids croissant, étape 1)
49     parent = []
50     rank = []
51
52
53     for node in range(Graph.V): # Cette boucle parcourt tous les sommets
    du graphe et crée un ensemble pour chacun d'entre eux
54         parent.append(node)
55         rank.append(0)
56
57     # Tant que le nombre d'arêtes est inférieur à V-1, notre sous-graphe
    n'atteint pas tous les sommets -> on continue
58     while e < Graph.V - 1:
59
60         u, v, w = Graph.graph[i] # self.graph contient les arêtes par
    ordre croissant de poids, on commence avec i = 0
61         i = i + 1 # puis à l'itération suivante on voudra
    avoir la 2ème arête la plus légère, donc on incrémente.
62
63         x = Graph.find(parent, u) # Ces 2 lignes des codes permettent de
    rechercher et de stocker à quel ensemble
64         y = Graph.find(parent, v) # appartiennent u et v.
65
66         if x != y: # Si u et v font déjà parti du Minimum Spanning Tree,
    i.e. u et v appartiennent au même ensemble
67             # Alors on ne veut pas ajouter cette arête au minimum
    spanning-tree, d'ou le x!=y
68             e = e + 1 # Si u et v sont d'ensemble différent, on a atteint
    un sommet de plus donc on incrémente
69             result.append([u, v, w]) # On ajoute la nouvelle arête au
    résultat
70             Graph.apply_union(parent, rank, x, y) # On fusionne l'ensemble
    auquel appartient v à celui auquel appartient u
71         for u, v, weight in result:
72             print("%d - %d: %d" % (u, v, weight)) # méthode permettant
    d'afficher le résultat
73
74     return result
75
76 kruskal_algo(g)

```

Question 5: (🕒 10 minutes) Social Network Analysis : Papier

Les graphes peuvent être utilisés pour représenter une multitude de choses. L'une d'entre elles est la représentation de votre réseau d'amis. Imaginez que vous possédiez une liste de vos amis ainsi que des amis de vos amis (qui ne sont pas nécessairement vos amis). Cette liste peut-être représentée sous forme de graphe. Dans ce graphe :

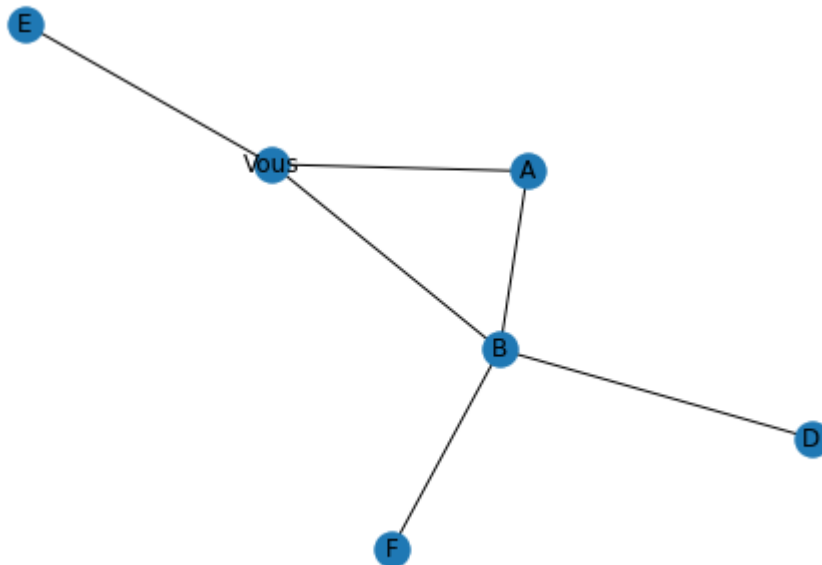
1. À quoi correspondent les arêtes et les nœuds ?
2. Faut-il utiliser un graphe dirigé ?

Supposez que vous disposiez d'un graphe des relations sociales. Décrivez comment retrouver les éléments suivants :

1. Votre ami qui a le plus d'ami
2. Découvrir quels amis à vous se connaissent
3. Listez vos amis qui pourraient vous présenter quelqu'un que vous ne connaissez pas (ami d'ami qui n'est pas votre ami)

Vous voudriez désormais ajouter une nouvelle personne sur ce graphe, mais cette dernière n'est ni votre ami, ni l'ami d'un de vos amis. Quel sera son degré dans le graphe ?

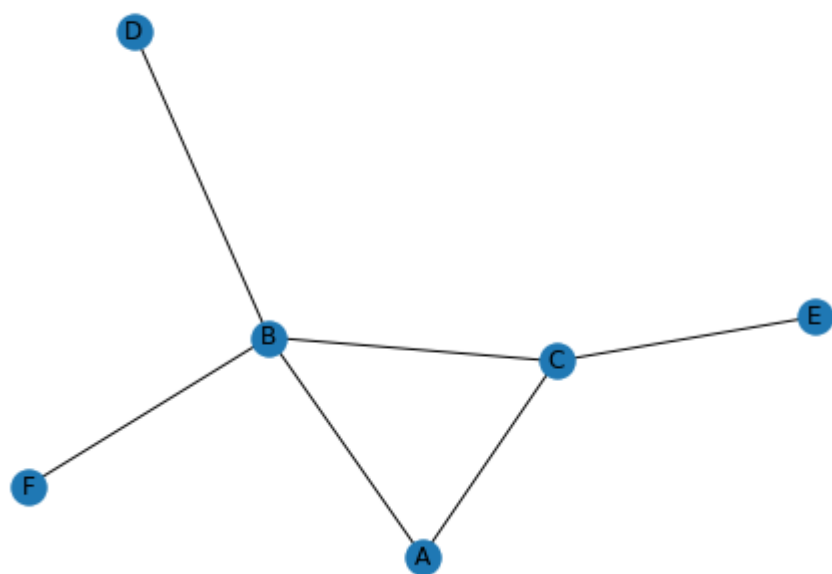
Retrouvez les éléments 1) à 3) dans le graphe ci-dessous :



Question 6: (🕒 5 minutes) Reconnaître des réseaux semblables

Supposons maintenant que vous travaillez chez Meta, qui a racheté Whatsapp, et que vous disposiez des 2 graphes représentant respectivement Facebook et Whatsapp. Pouvez-vous déterminer visuellement à qui correspondent les individus de Facebook sur Whatsapp ?

Graphe de Facebook :



Graphe de Whatsapp :

